

Análisis de programas vulnerables

Laura de Haro García

Análisis de los resultados

Como ya se ha mencionado previamente, en el presente trabajo se han analizado 4 grandes modelos de lenguaje, y más concretamente, su capacidad de detectar vulnerabilidades en los 2 contratos inteligentes definidos mediante el uso de 2 prompts diferentes.

La capacidad de detectar la presencia de vulnerabilidades

En esta sección se analizará la capacidad de los modelos y los prompts de detectar la presencia de las vulnerabilidades en el código. Como ya se ha establecido previamente, un contrato puede presentar la misma vulnerabilidad en más de una función, sin embargo, en esta parte del análisis, la repetición de la misma vulnerabilidad no se tendrá en cuenta. En otras palabras, si un contrato tiene la vulnerabilidad “A” x veces, no se apunta el número de veces que aparece, únicamente que está presente. Igualmente, no interesa en cuántas funciones el modelo detecta la presencia de la vulnerabilidad “A”, únicamente interesa si ha detectado su presencia en al menos una. Por lo tanto, si un contrato tiene 2 veces una vulnerabilidad de *underflow* y el modelo únicamente menciona una de sus apariencias, se considera que ha detectado correctamente la presencia de la vulnerabilidad.

Por lo tanto, para cada vulnerabilidad se crean dos vectores: Uno que señala la presencia o ausencia de la vulnerabilidad en los contratos y otro que señala si el modelo en cuestión la había detectado o no. Podemos utilizar estos dos vectores para crear una matriz de confusión.

```
xtabs(formula = ~ PDACheckPredicted + PDACheckReal, data = gptNanoComplexWalletAny)
```

	PDACheckReal	
PDACheckPredicted	TRUE	FALSE
TRUE	14	2
FALSE	19	28

En el caso concreto representado en la imagen, es decir, el modelo **gpt-nano-4.1** con el prompt **complejo** analizando la vulnerabilidad donde se omite el control de verificación del PDA de una cuenta en el programa **Wallet**, podemos observar 14 detecciones correctas de la vulnerabilidad (verdaderos positivos), 28 detecciones correctas de un contrato sin la vulnerabilidad (verdaderos negativos), pero también 19 falsos negativos y 2 falsos positivos. A partir de estos datos, podemos calcular varias métricas que nos permiten diagnosticar la calidad de la clasificación. En este trabajo, se empleará la precisión, la sensibilidad y la especificidad.

La precisión sería simplemente:

$$precisin = \frac{\text{número de predicciones correctas}}{\text{número de predicciones}}$$

La sensibilidad se calcularía de la siguiente manera:

$$sensibilidad = \frac{\text{número de verdaderos positivos}}{\text{número de verdaderos positivos} + \text{número de falsos negativos}}$$

En otras palabras, la sensibilidad comunica qué porcentaje de contratos vulnerables han sido identificados identificados como vulnerables.

Finlamente, la especificidad se calcularía de la siguiente manera:

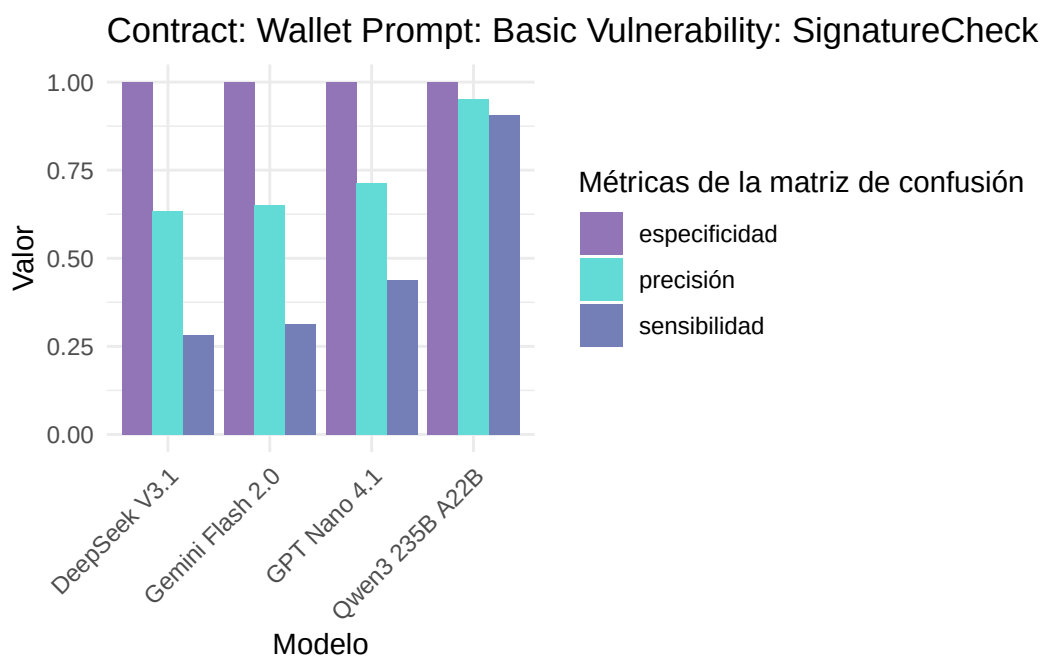
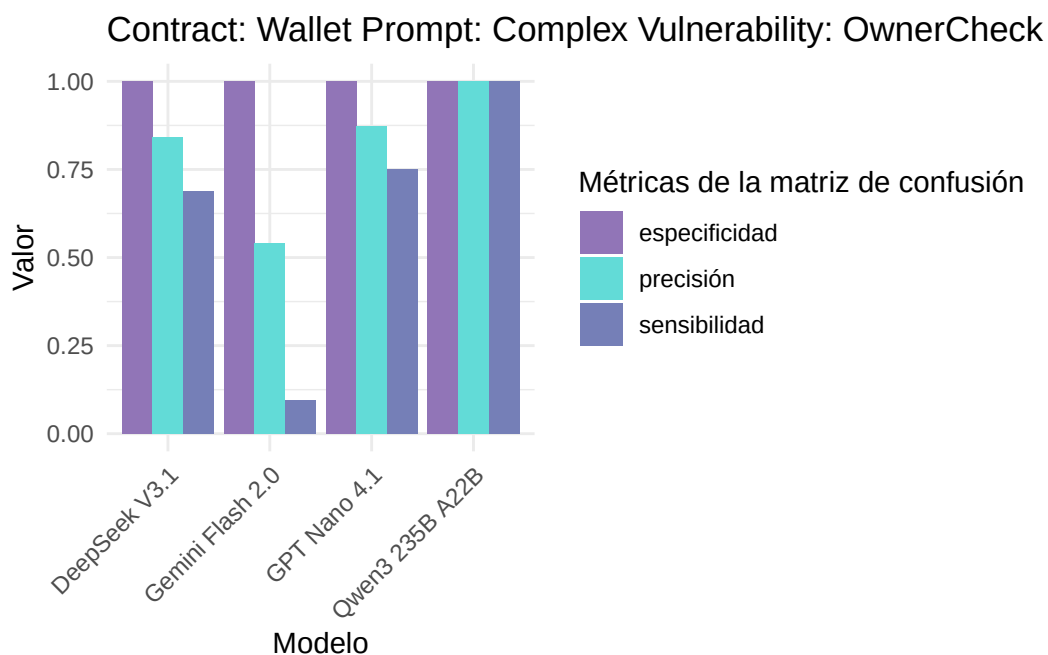
$$especificidad = \frac{\text{número de verdaderos negativos}}{\text{número de verdaderos negativos} + \text{número de falsos positivos}}$$

En otras palabras, la sensibilidad comunica qué porcentaje de contratos seguros han sido identificados como seguros.

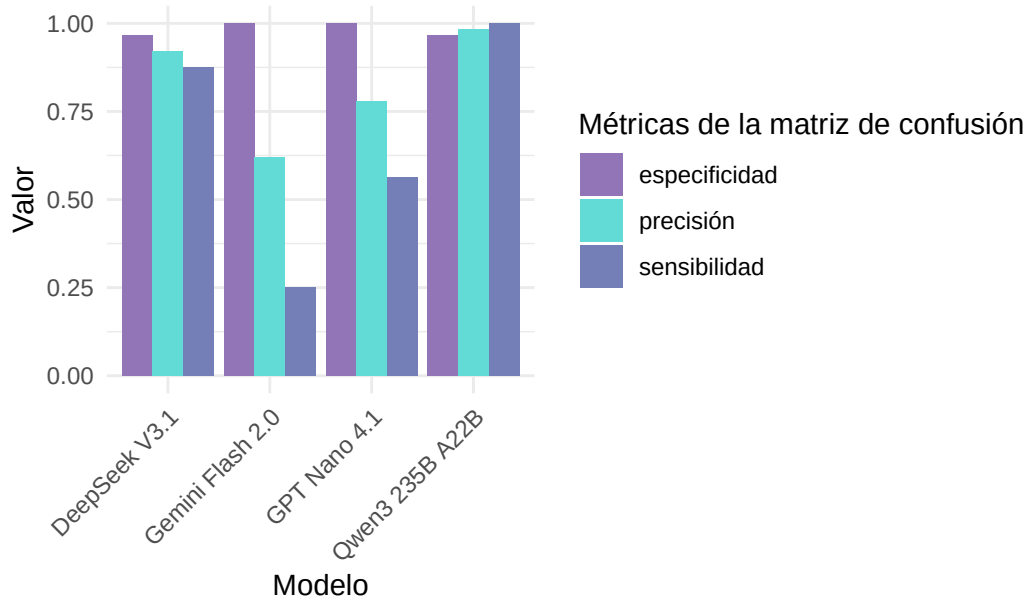
En el caso de la matriz de confusión anteriormente expuesta, su precisión equivaldría a 66.6%, la sensibilidad a 42.42% y la especificidad a ~93.3%.

De la misma manera, podemos evaluar el rendimiento de todos los modelos y prompts en cada una de las vulnerabilidades, primero en el contrato **Wallet**:

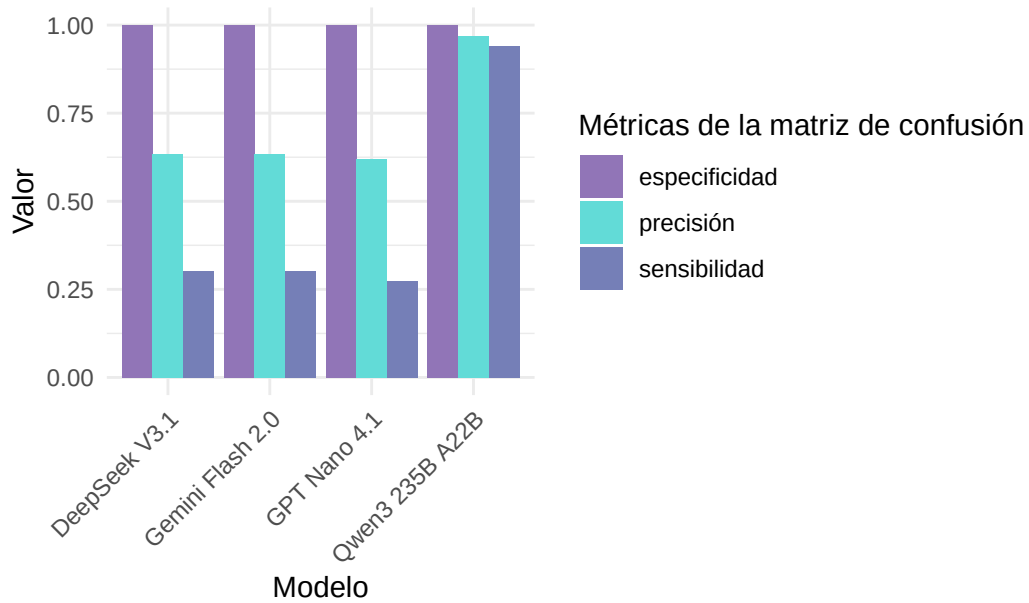
```
Warning: `data_frame()` was deprecated in tibble 1.1.0.
i Please use `tibble()` instead.
```

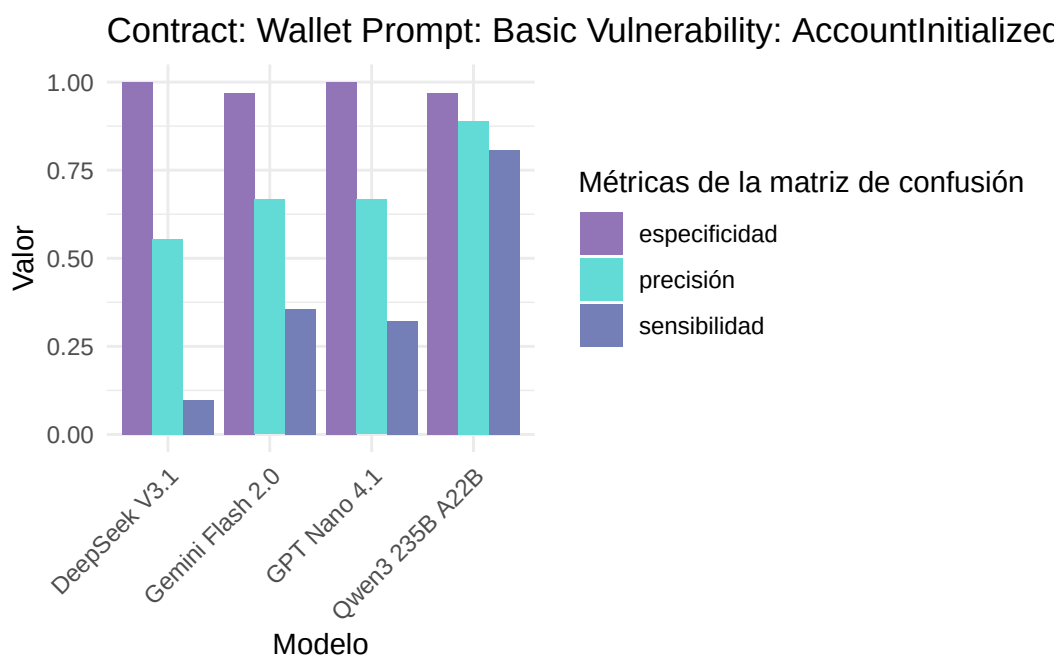
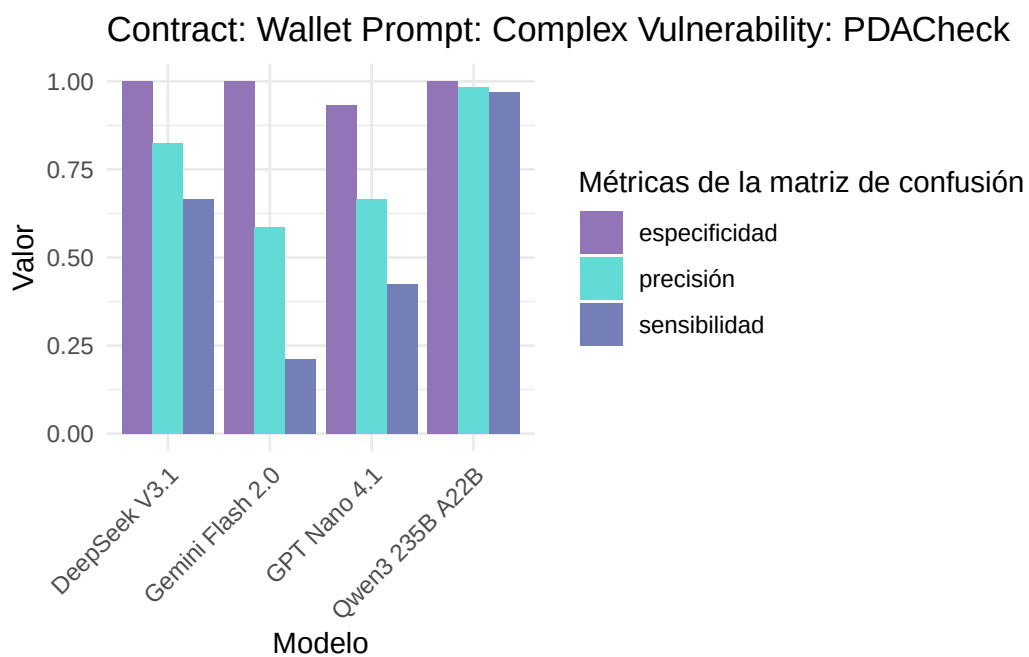


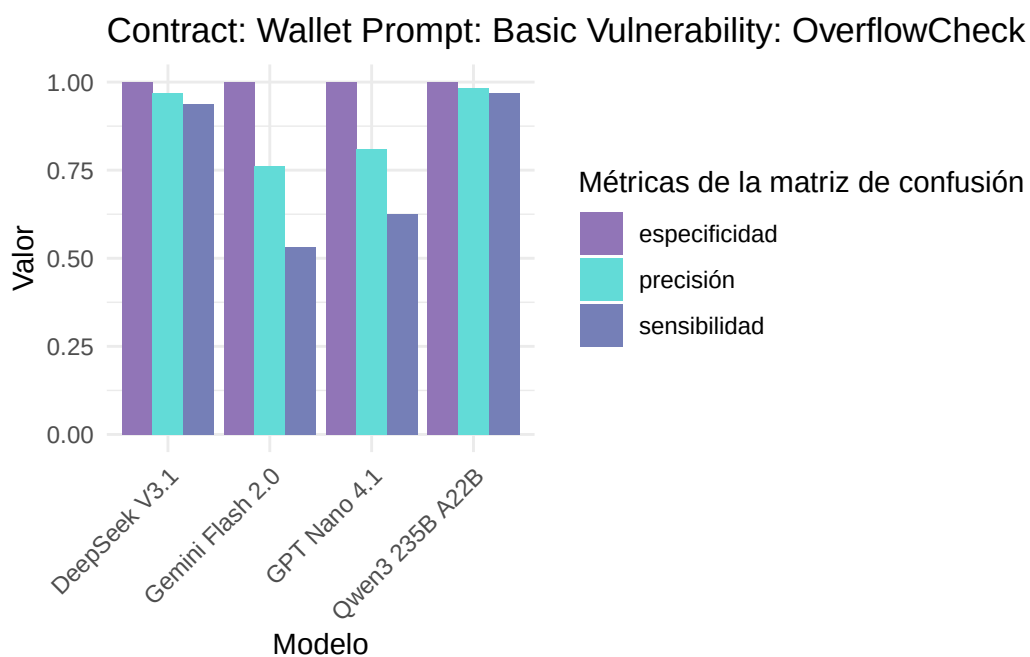
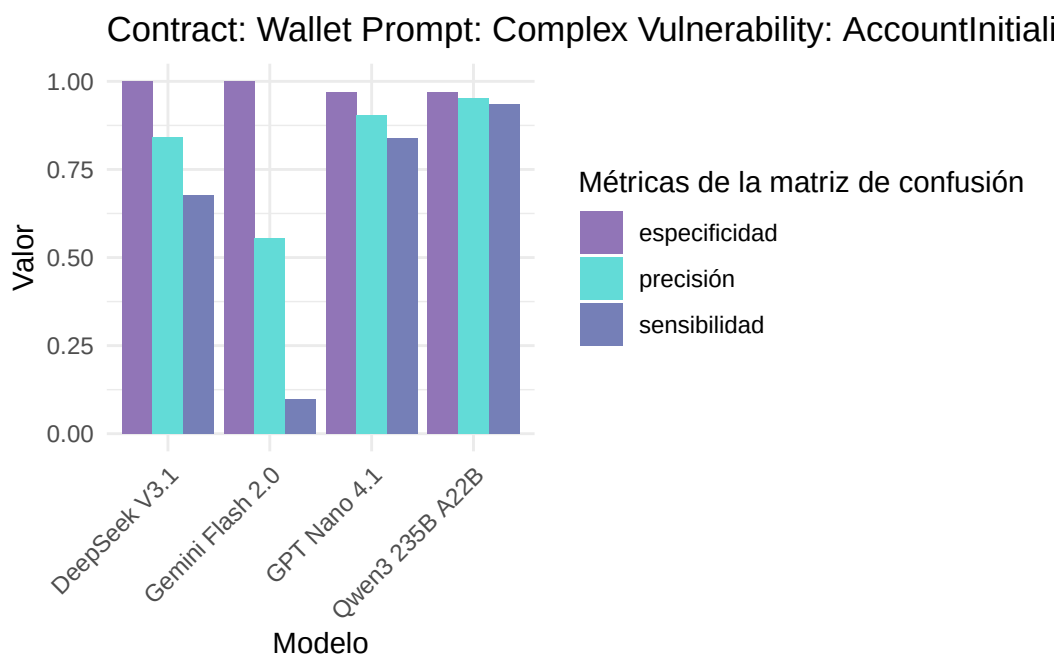
Contract: Wallet Prompt: Complex Vulnerability: SignatureChe

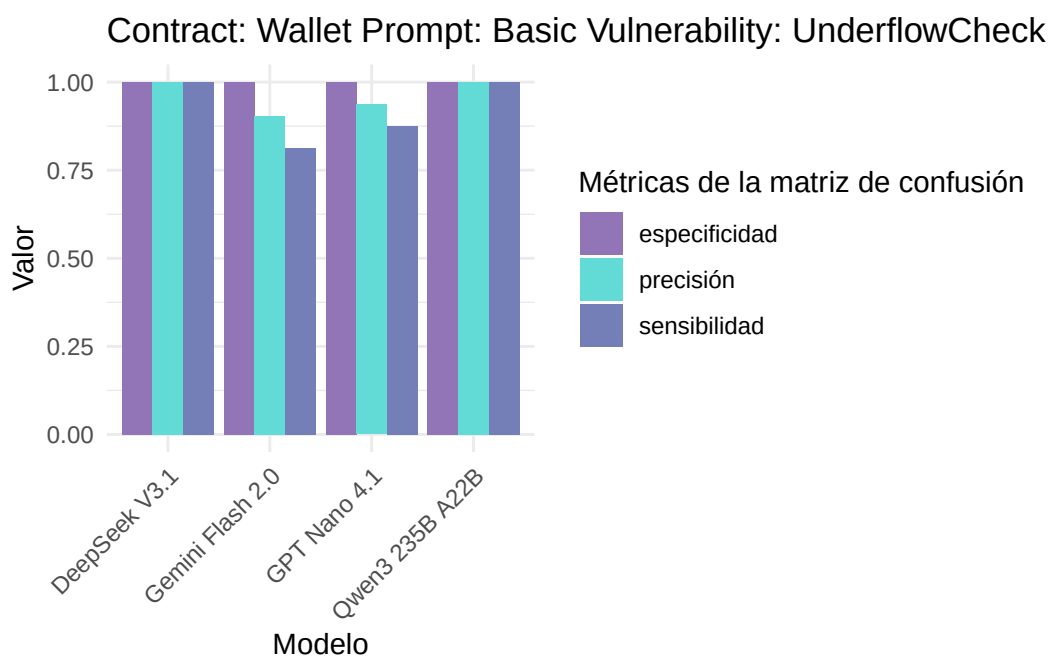
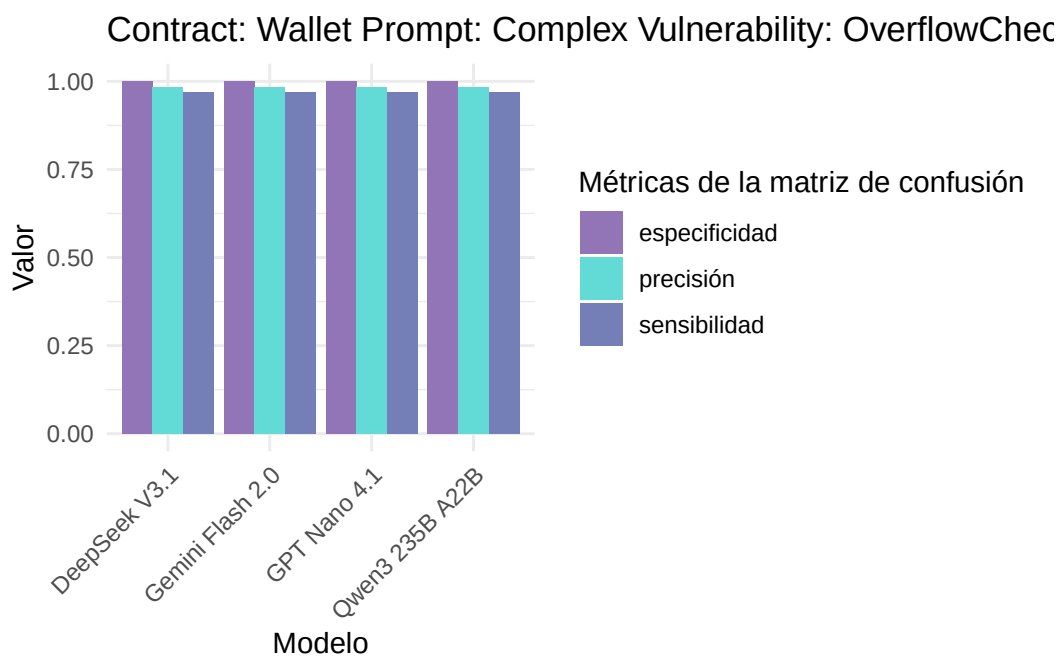


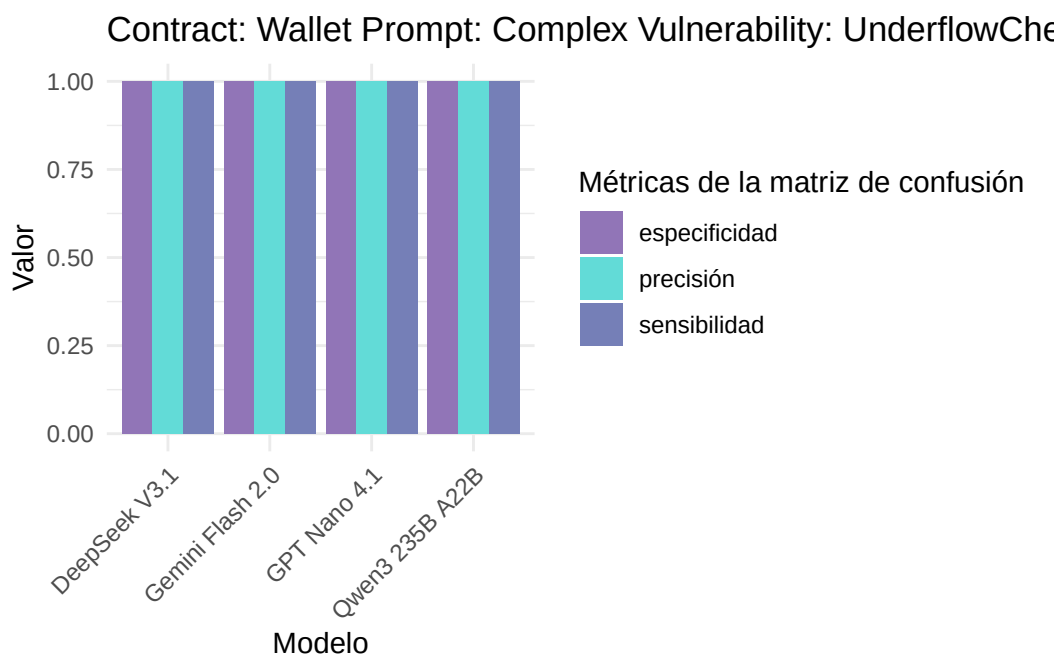
Contract: Wallet Prompt: Basic Vulnerability: PDACheck



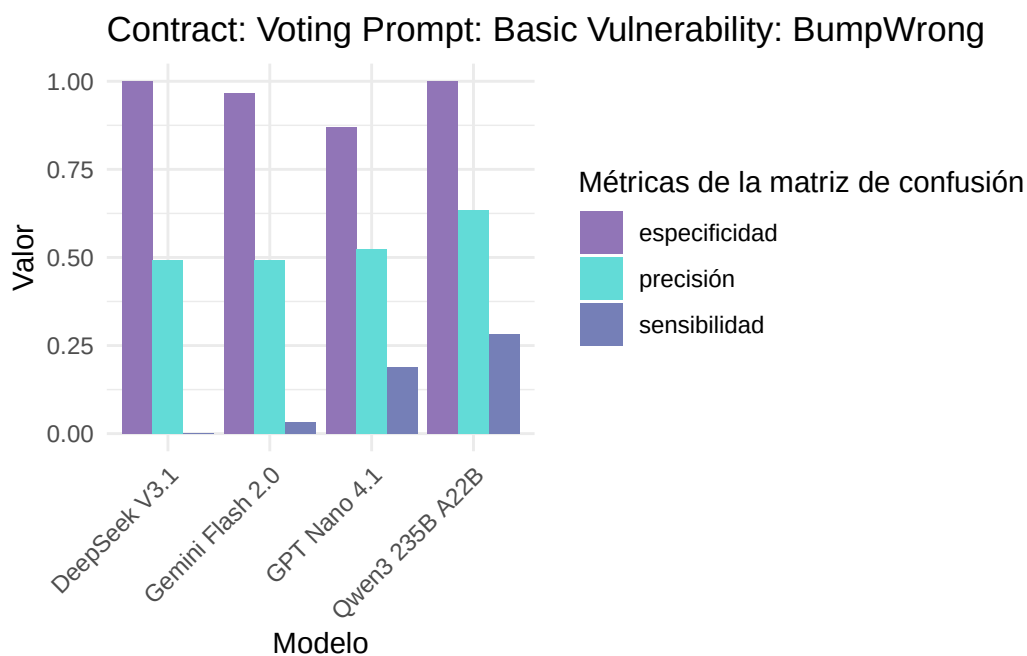


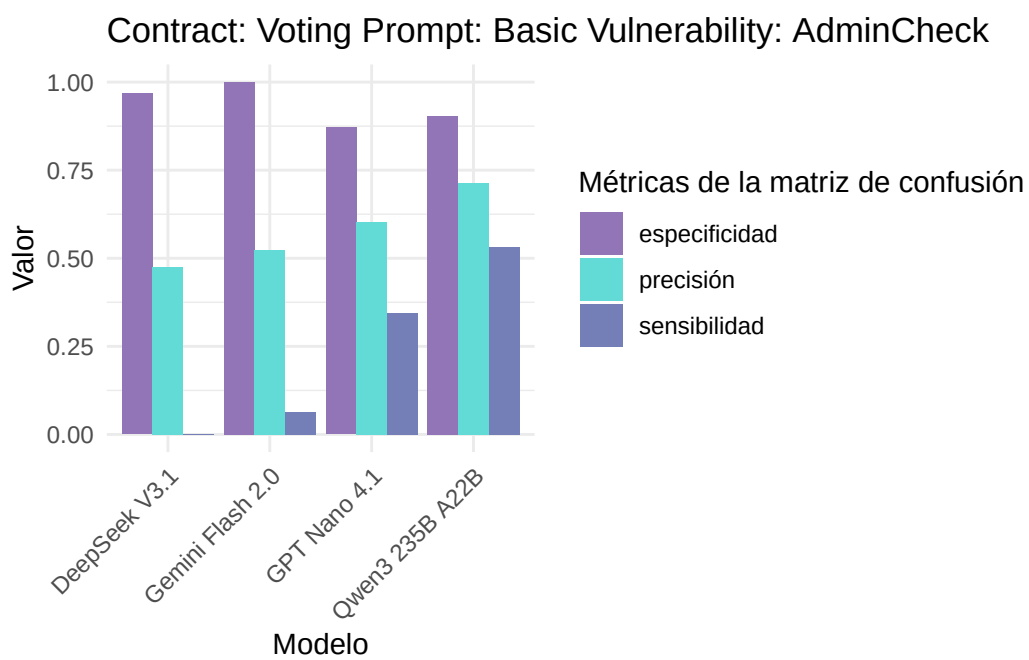
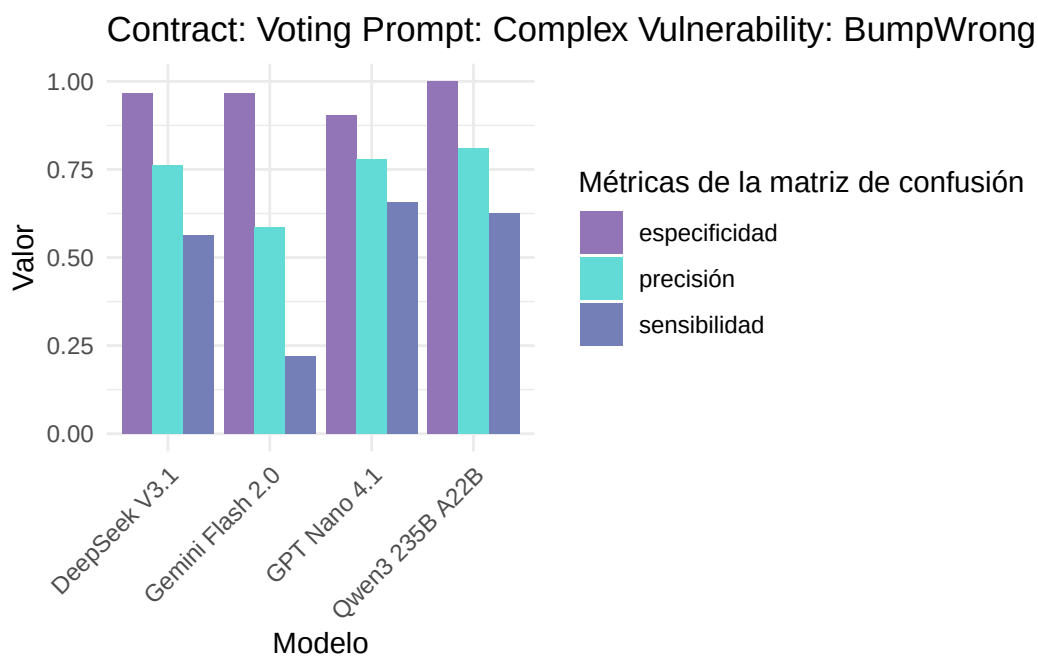


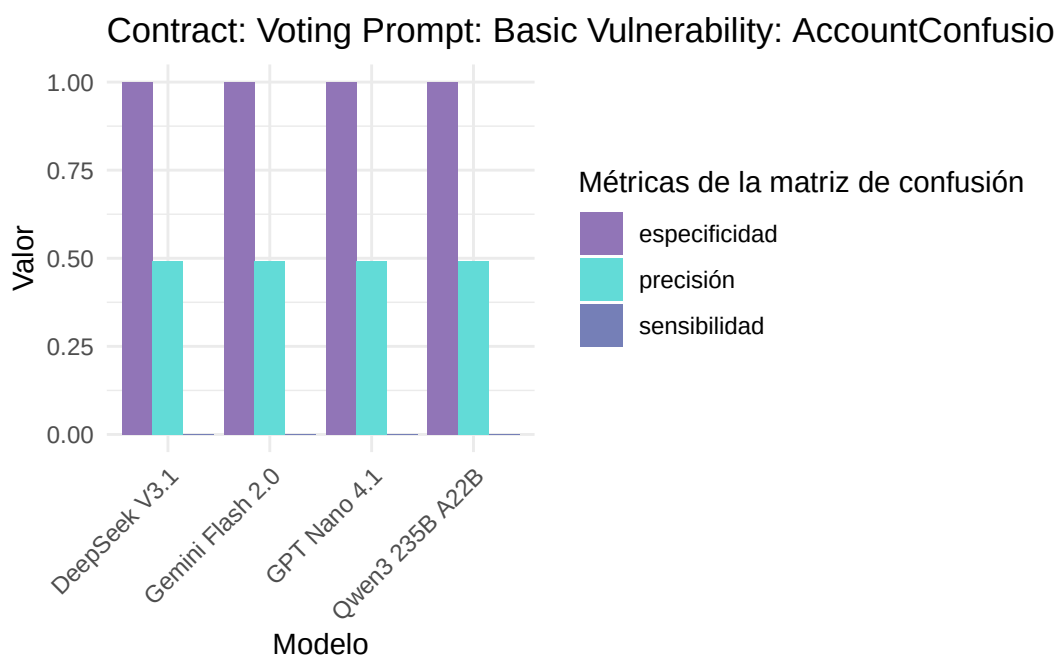
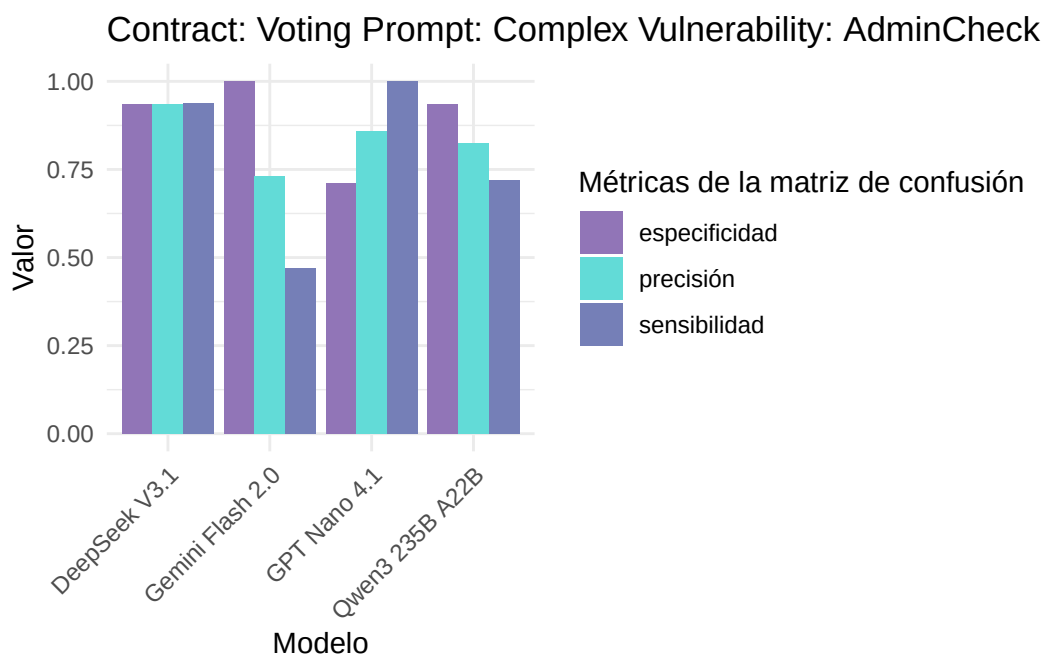


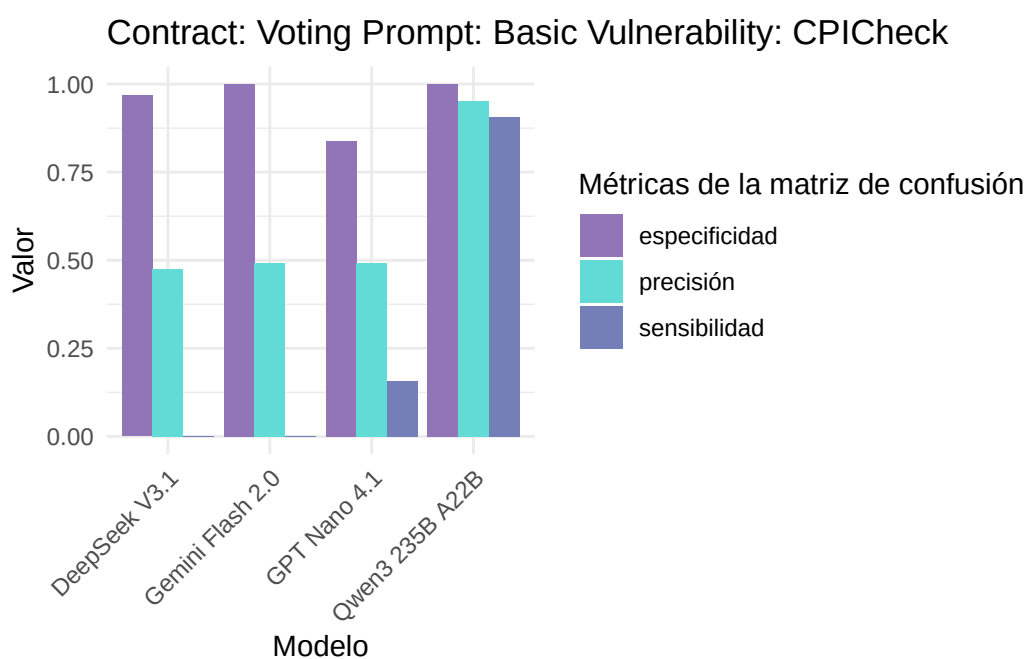
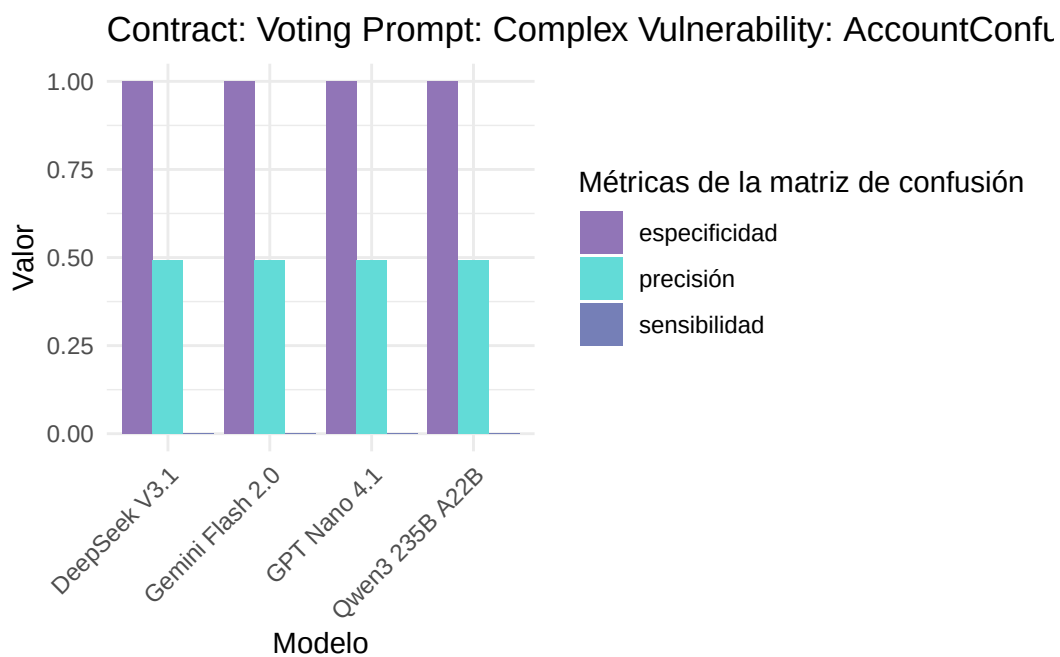


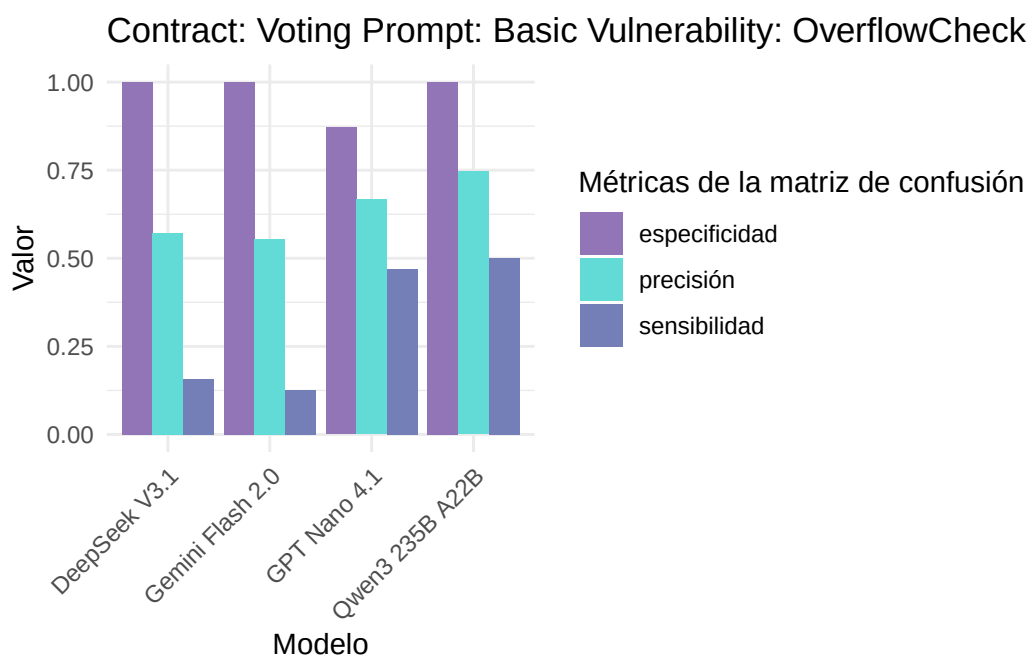
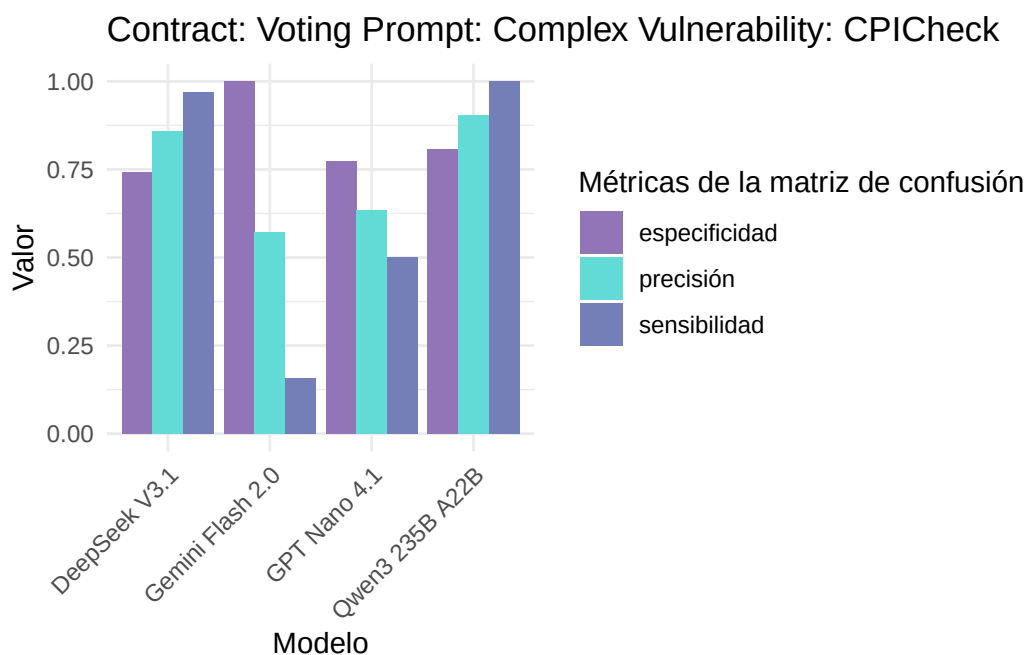
E igualmente, en el contrato **Voting**.

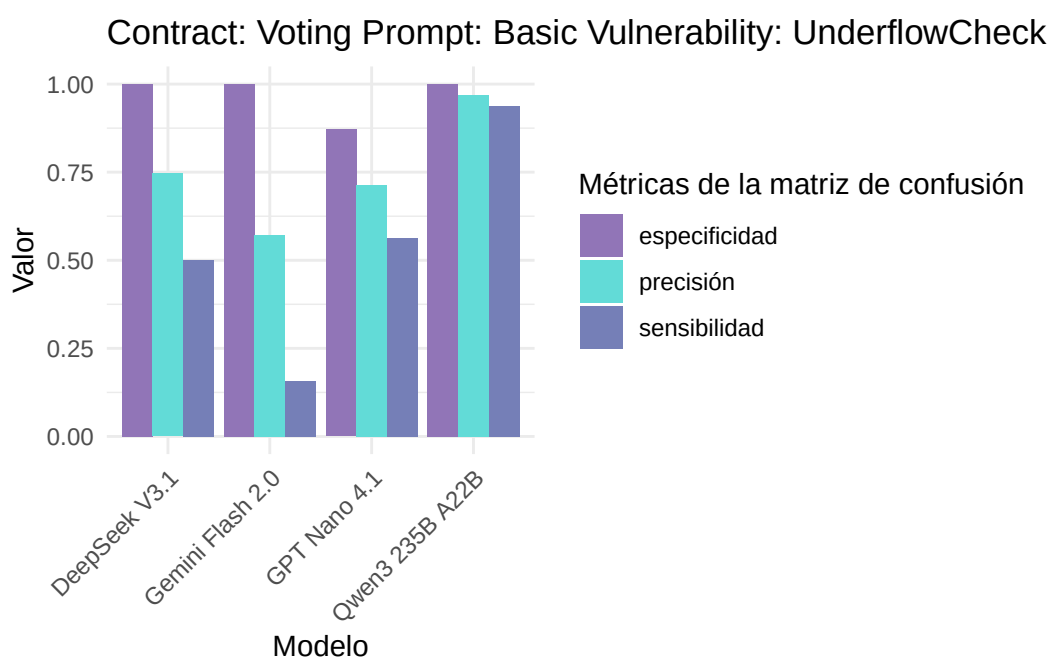
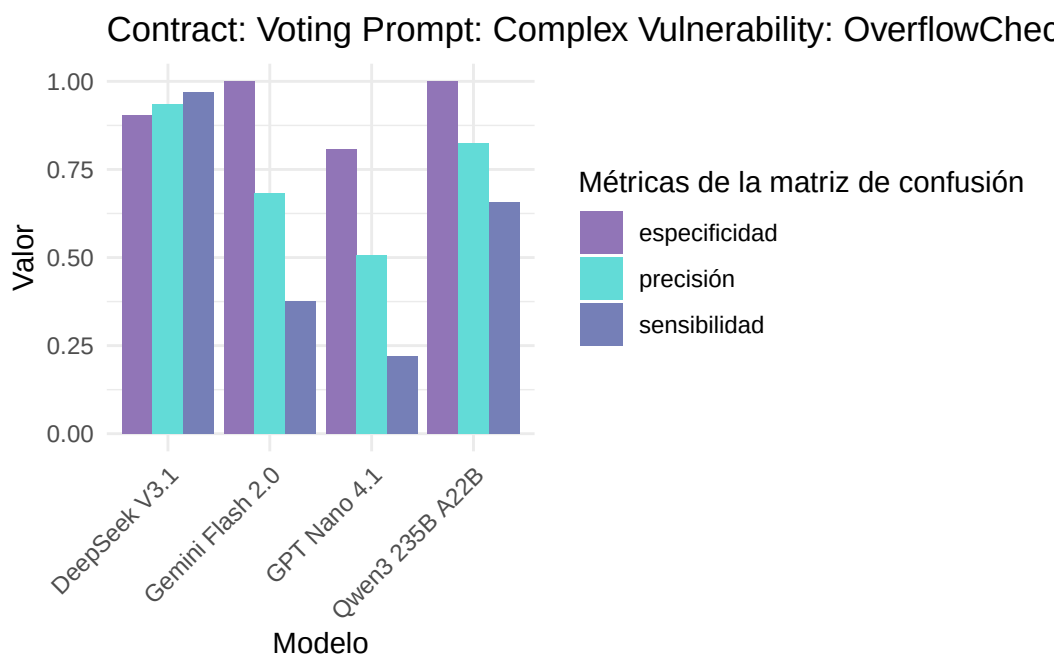


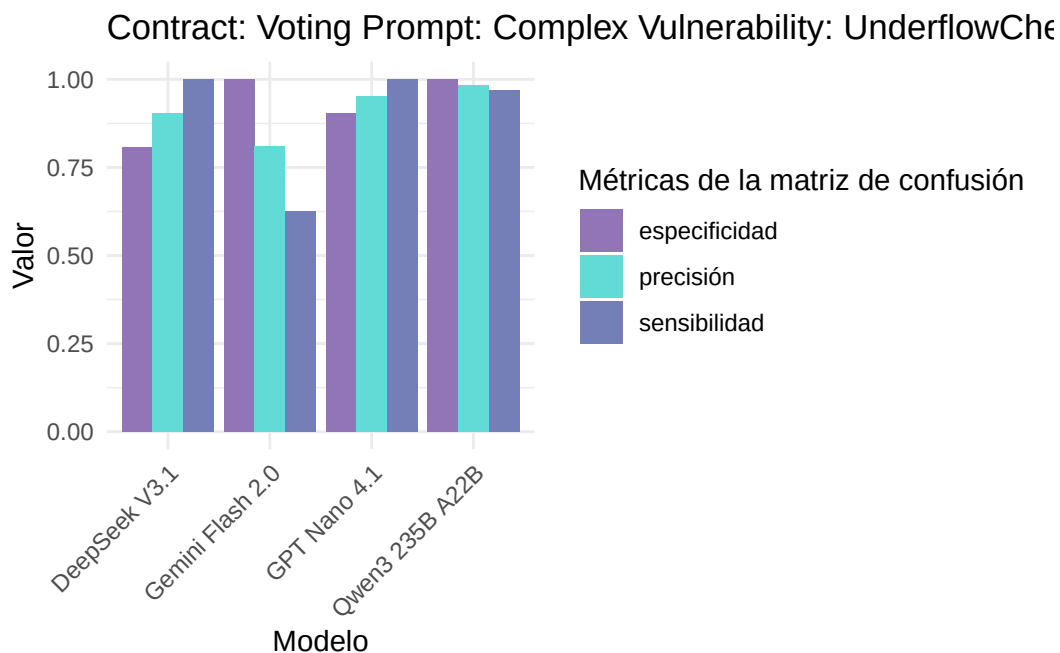








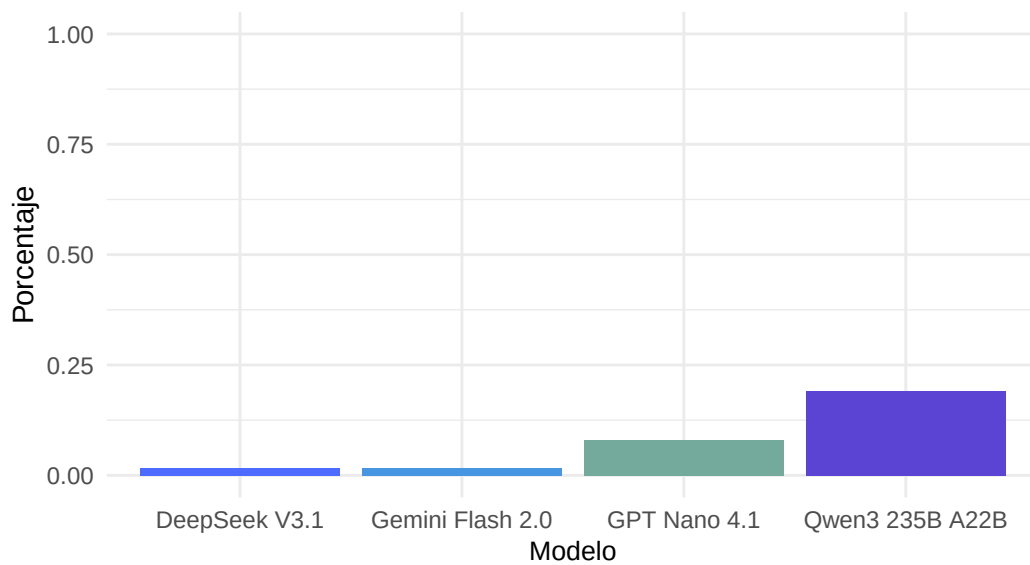




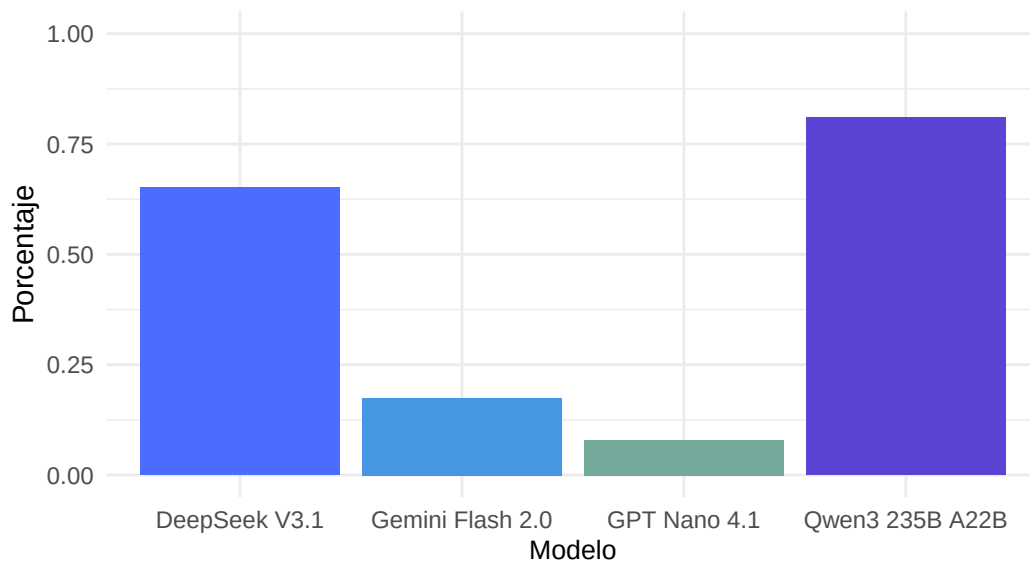
Detección de la función deprecada

Finalmente también cabe mencionar que todos los contratos **Voting** contaban con una función deprecada. Ya que no se trata de una vulnerabilidad prototípica, no se ha creado una alternativa no vulnerable, sino que solamente interesa en qué porcentaje de los contratos los modelos llegaron a señalar la presencia de una función deprecada.

Porcentaje de programas en los que cada modelo detectó la fi
Prompt básico



Porcentaje de programas en los que cada modelo detectó la fi
Prompt complejo



Capacidad de detección del número correcto de vulnerabilidades

En este apartado, se examinará la cantidad de vulnerabilidades que detectan los diferentes modelos y prompts. Los datos se examinarán mediante el error medio absoluto (MAE), raíz del error cuadrático medio (RMSE) y el sesgo.

El error medio absoluto se utilizará para ver la distancia media entre el número real de vulnerabilidades y el número predicho. Se puede calcular utilizando la siguiente fórmula:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Donde n es el número de casos totales, y es el número de errores real e $\hat{y}$ es el número de errores predicho.

La raíz del error cuadrático medio se utiliza sobre todo para penalizar los errores más grandes en los que la predicción se aleja en gran medida del número real. Aunque a nivel descriptivo no sea la medida más relevante, se utiliza frecuentemente como función de pérdida y por lo tanto, podría emplearse para tunear un modelo especializado. Podemos conseguir su valor utilizando la siguiente fórmula:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Otra vez, n es el número de casos totales, y es el número de errores real e $\hat{y}$ es el número de errores predicho

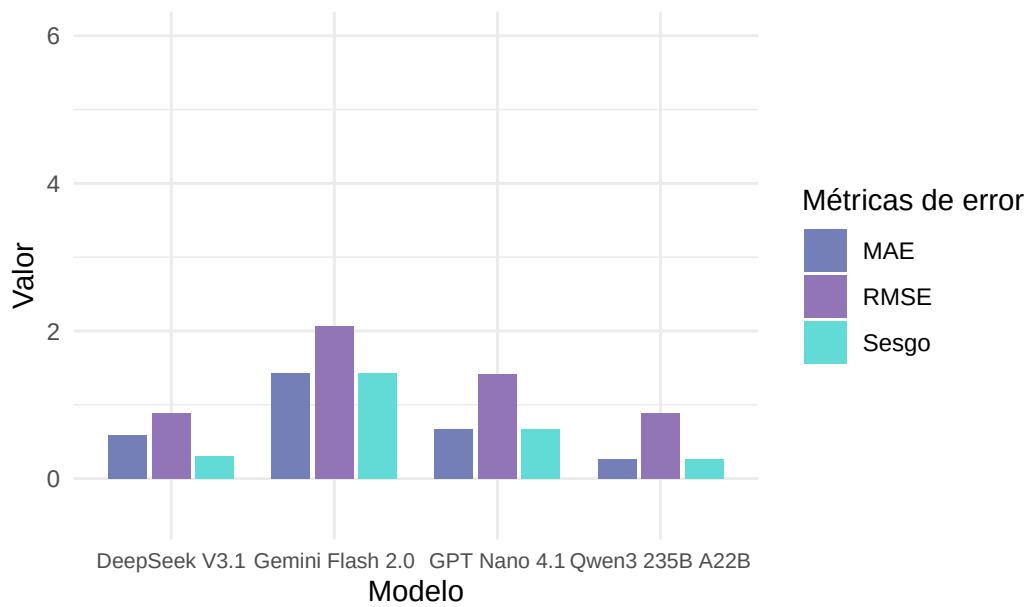
Finalmente, el sesgo se empleará para ver la dirección del error, es decir, si el modelo predice más vulnerabilidades de las que hay (sesgo negativo) o menos de las que hay (sesgo positivo).

$$\text{Sesgo} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

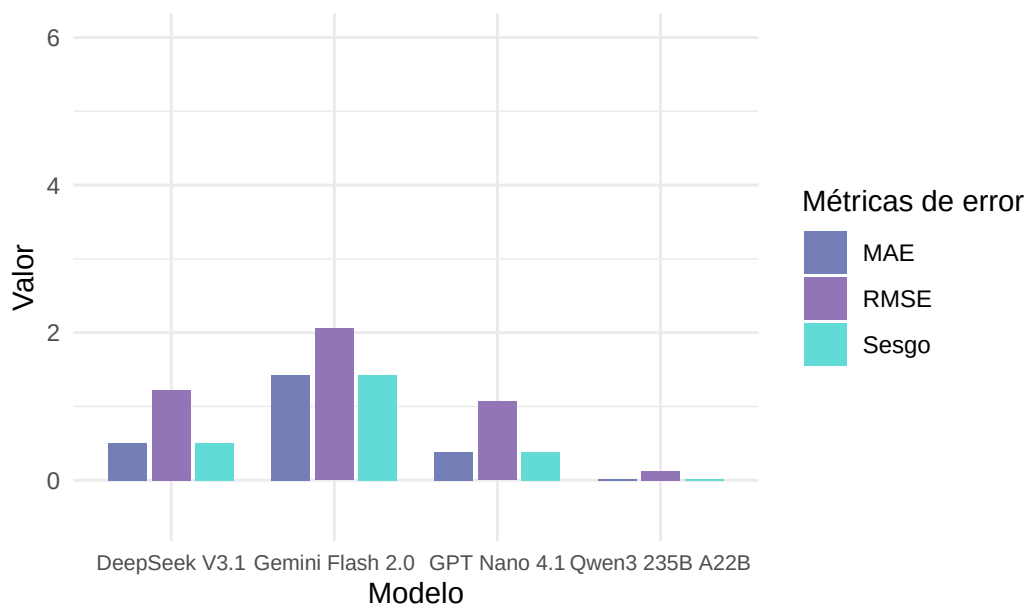
Igualmente que en los casos anteriores, n es el número de casos totales, y es el número de errores real e $\hat{y}$ es el número de errores predicho

Podemos examinar los errores en el contrato **Wallet**

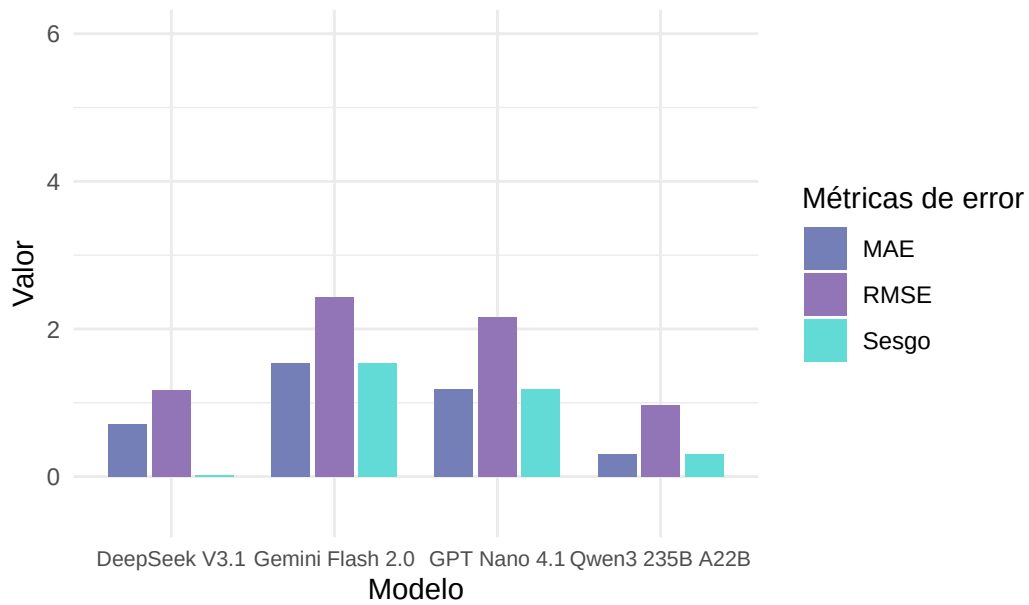
Contract: Wallet Prompt: Basic Vulnerability: OwnerCheck



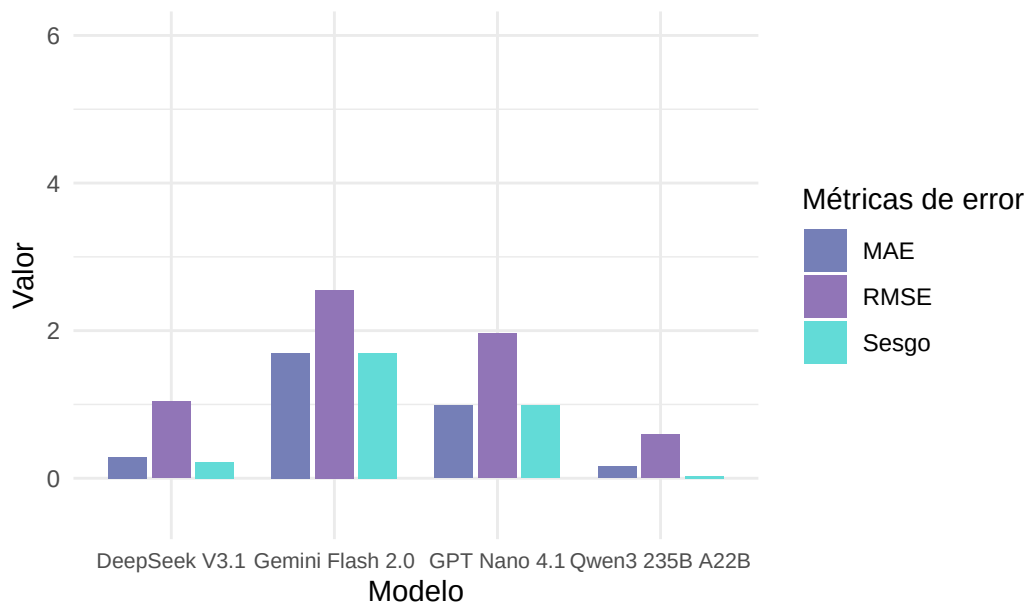
Contract: Wallet Prompt: Complex Vulnerability: OwnerCheck



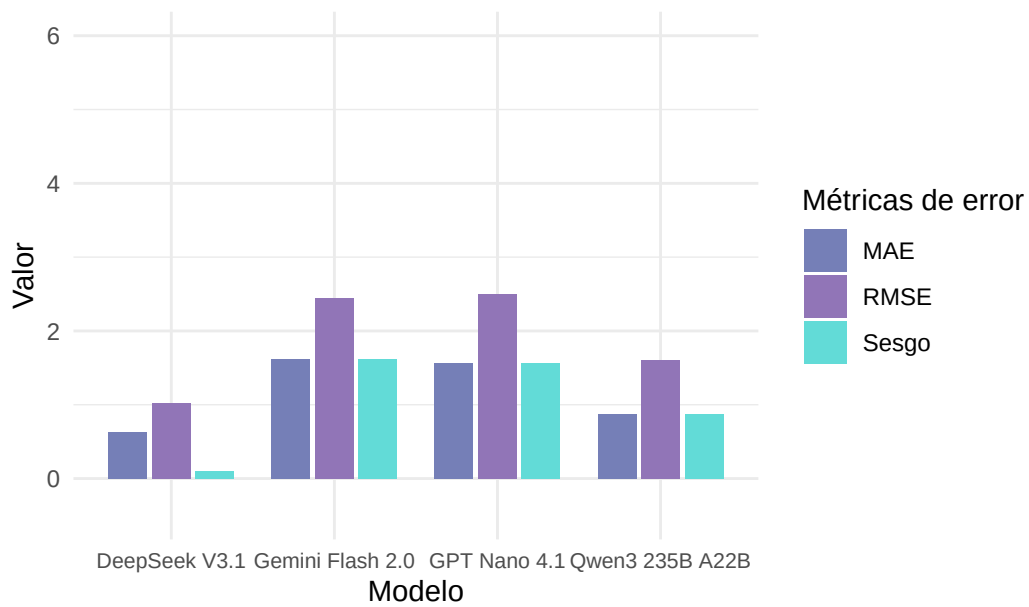
Contract: Wallet Prompt: Basic Vulnerability: SignatureCheck



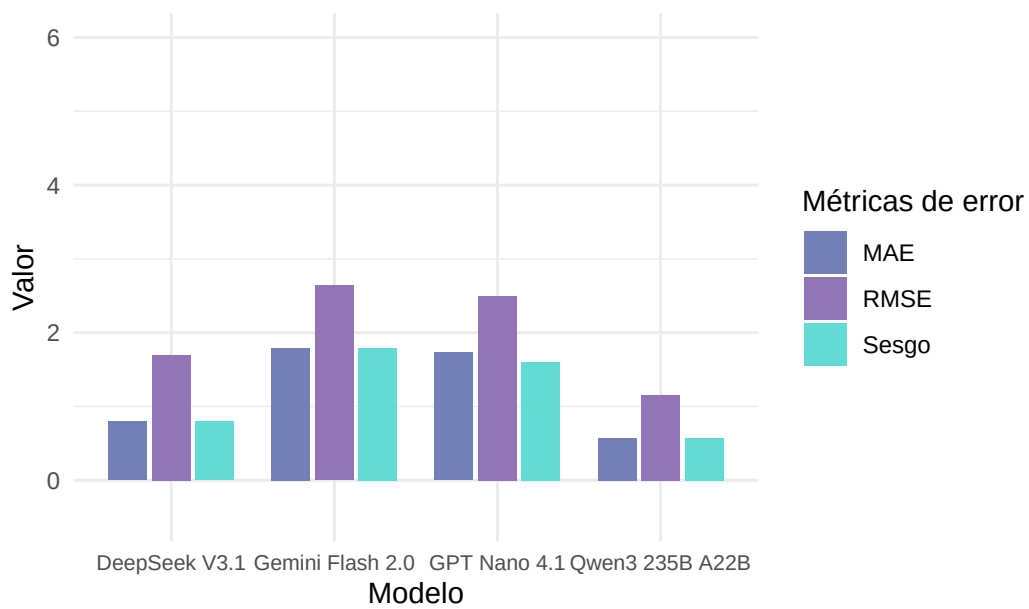
Contract: Wallet Prompt: Complex Vulnerability: SignatureCheck



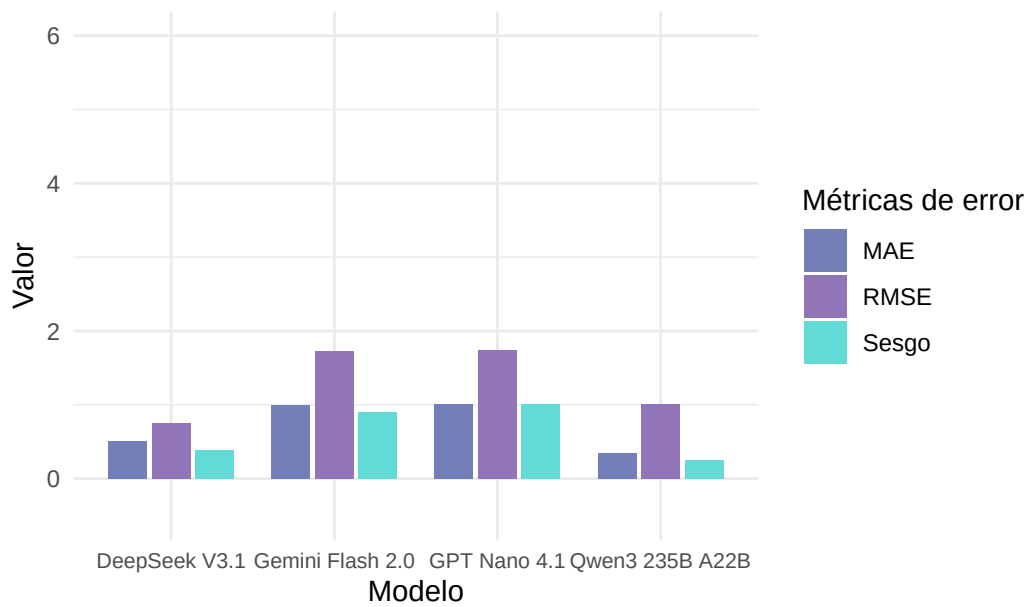
Contract: Wallet Prompt: Basic Vulnerability: PDACheck



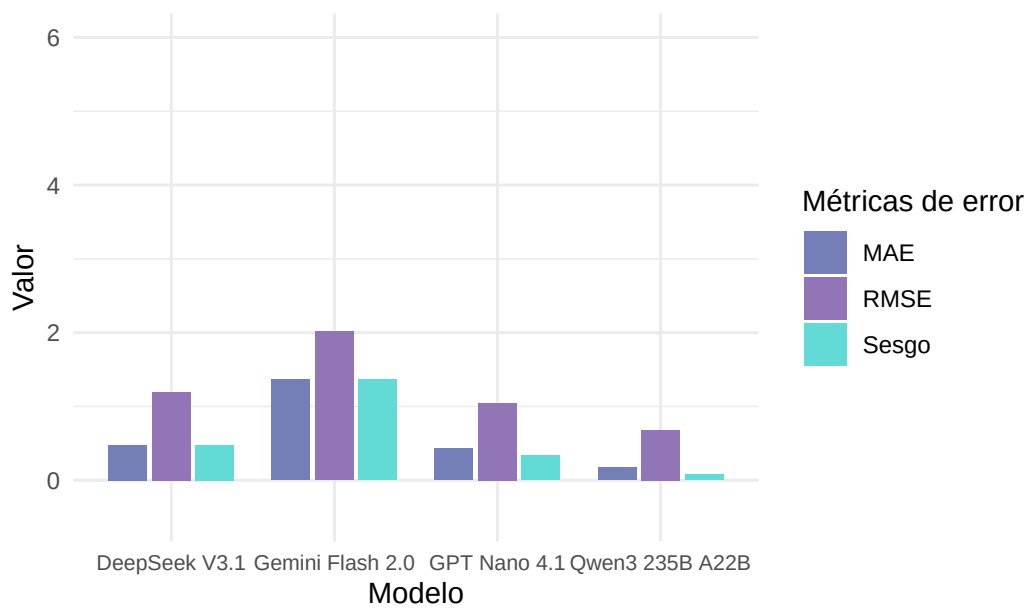
Contract: Wallet Prompt: Complex Vulnerability: PDACheck



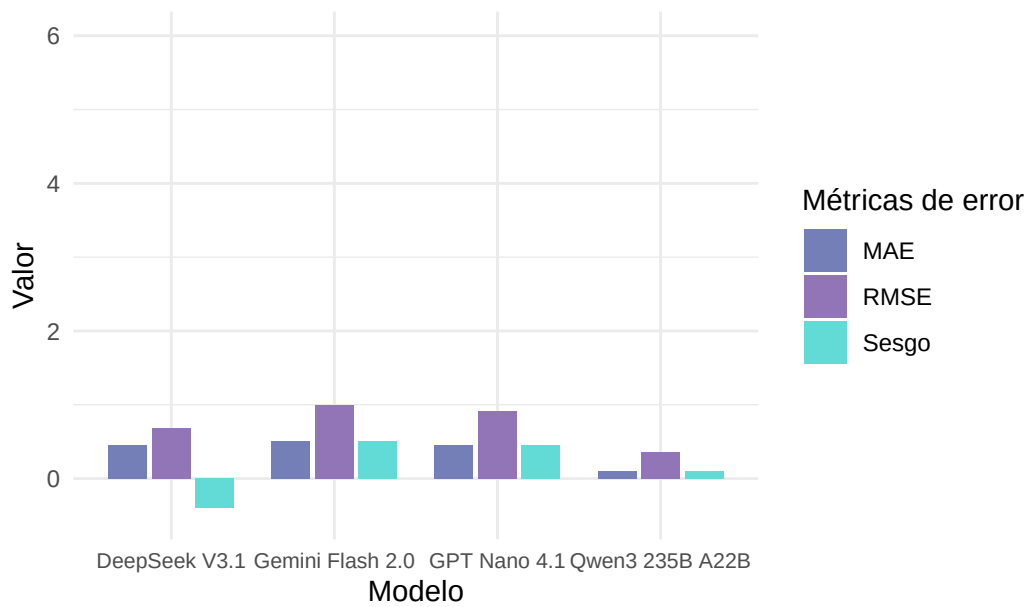
Contract: Wallet Prompt: Basic Vulnerability: AccountInitializedC



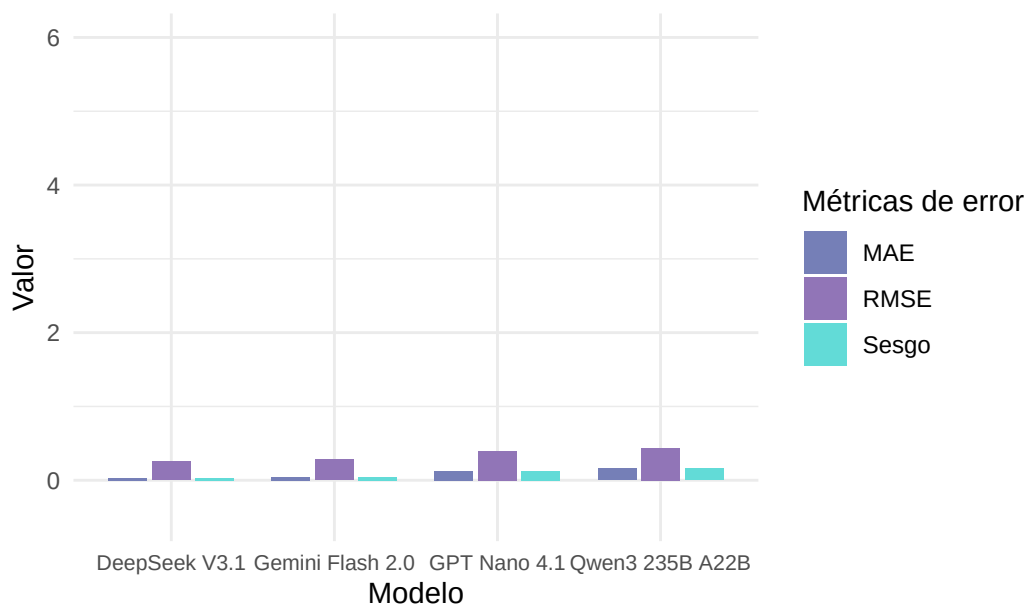
Contract: Wallet Prompt: Complex Vulnerability: AccountInitialize



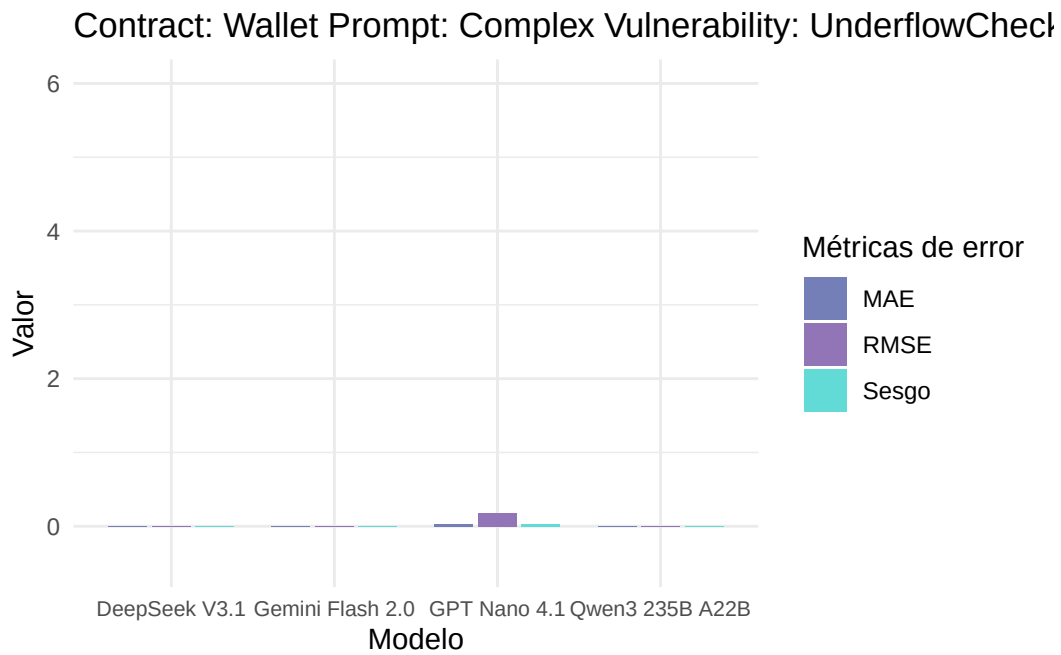
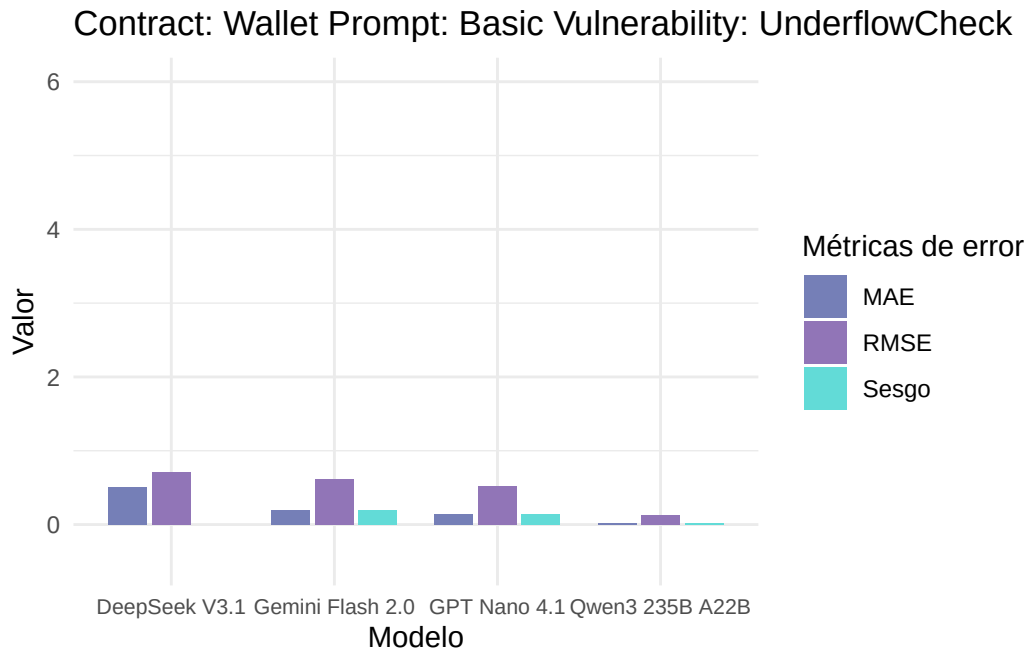
Contract: Wallet Prompt: Basic Vulnerability: OverflowCheck



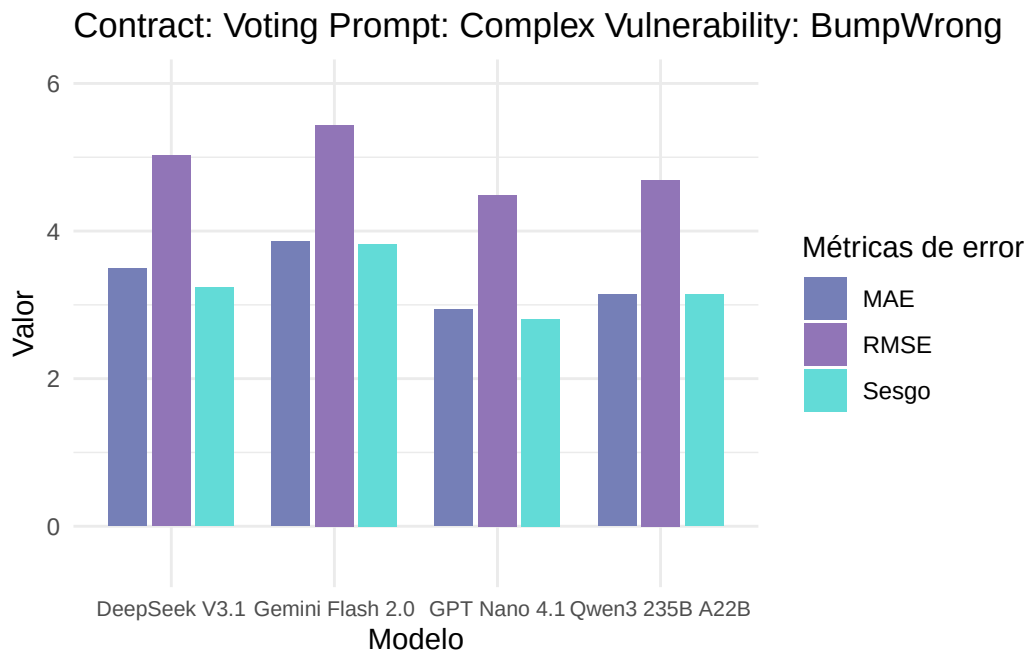
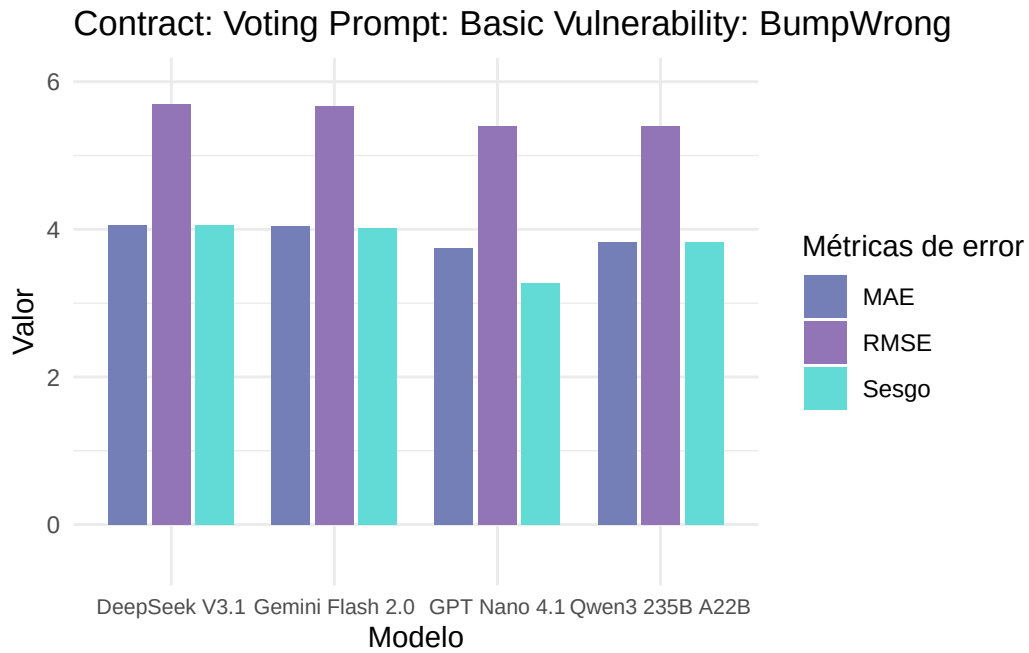
Contract: Wallet Prompt: Complex Vulnerability: OverflowCheck



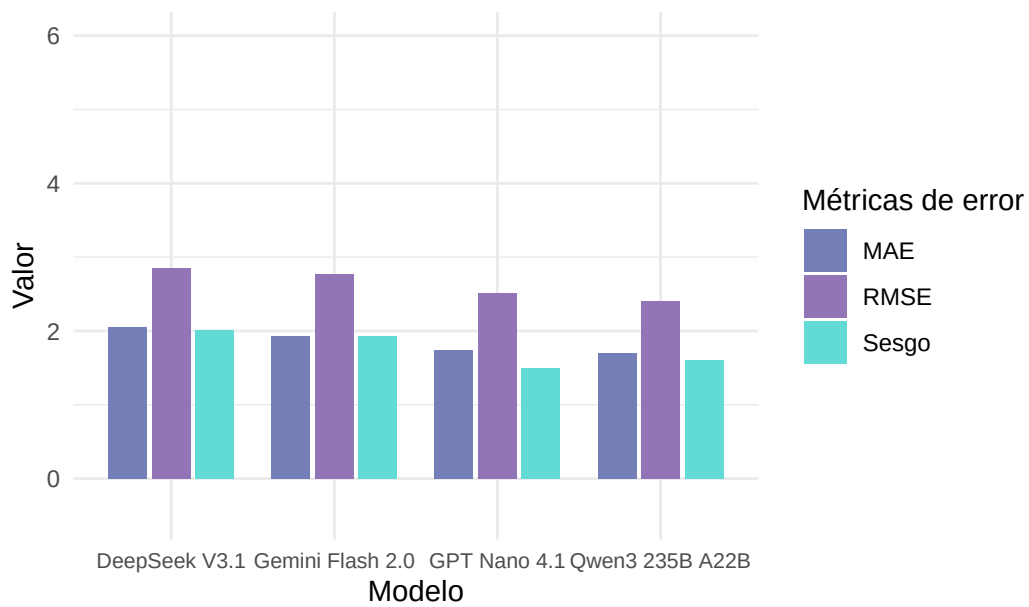
Warning: Removed 1 row containing missing values or values outside the scale range (``geom_col()``).



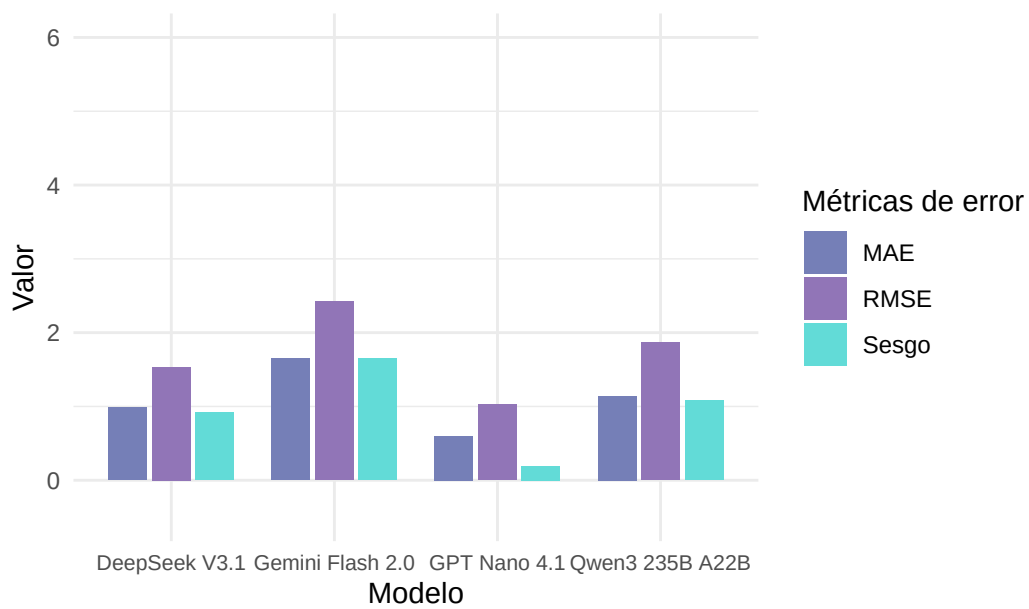
Y de la misma manera, en el contrato **Voting**.



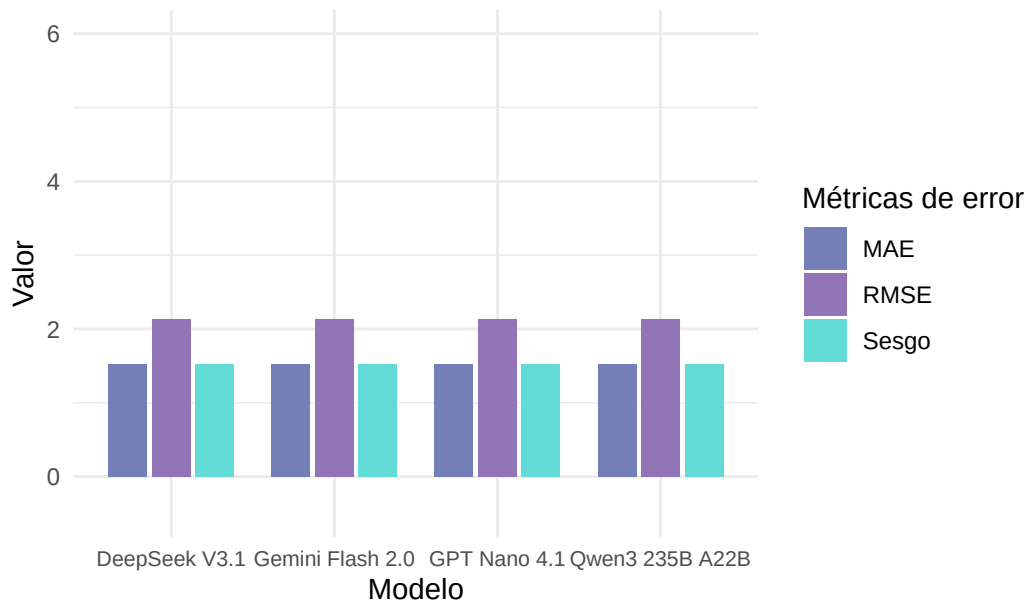
Contract: Voting Prompt: Basic Vulnerability: AdminCheck



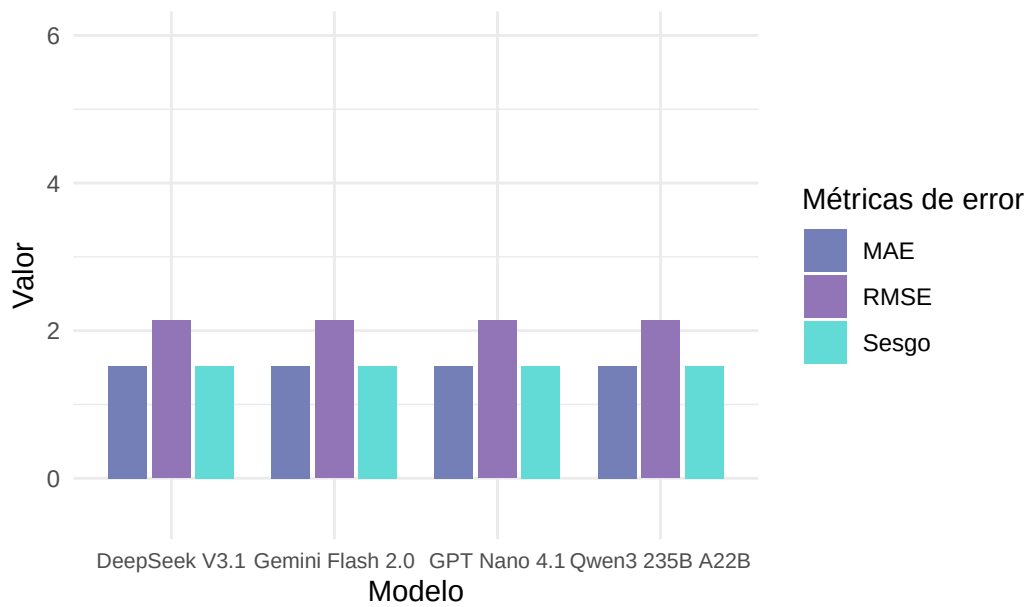
Contract: Voting Prompt: Complex Vulnerability: AdminCheck



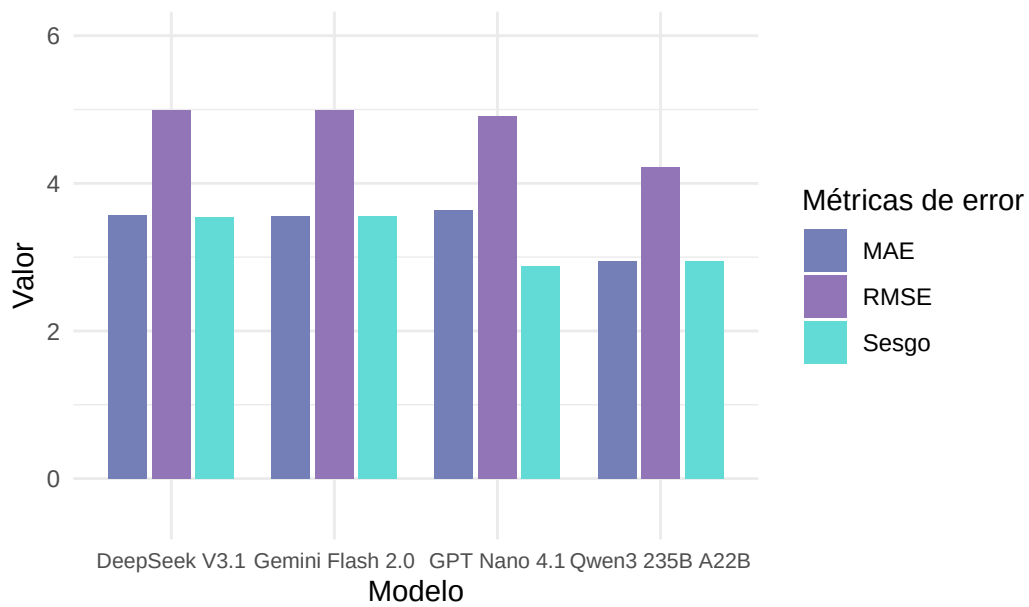
Contract: Voting Prompt: Basic Vulnerability: AccountConfusionC



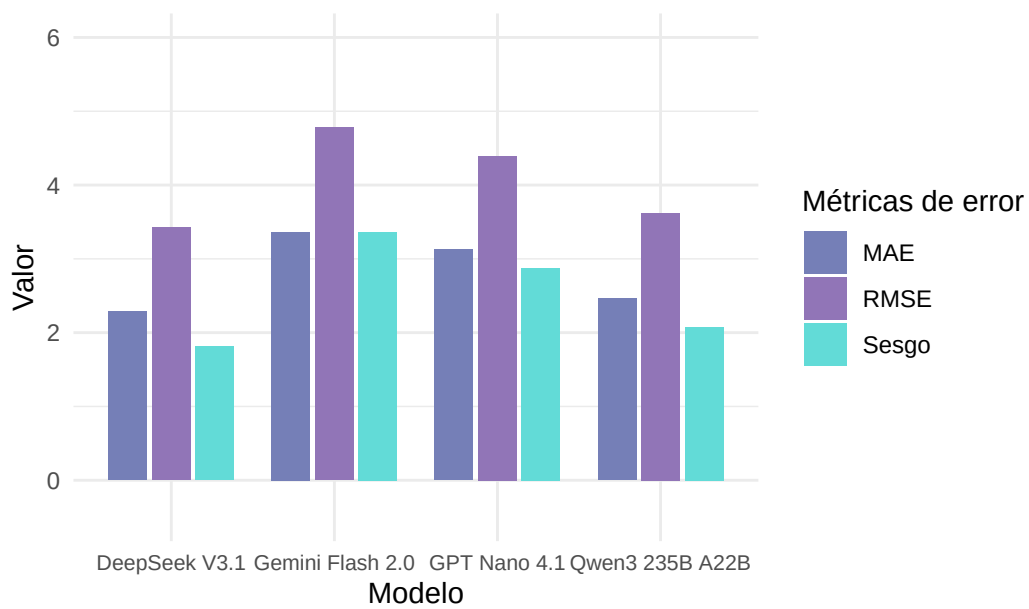
Contract: Voting Prompt: Complex Vulnerability: AccountConfusi



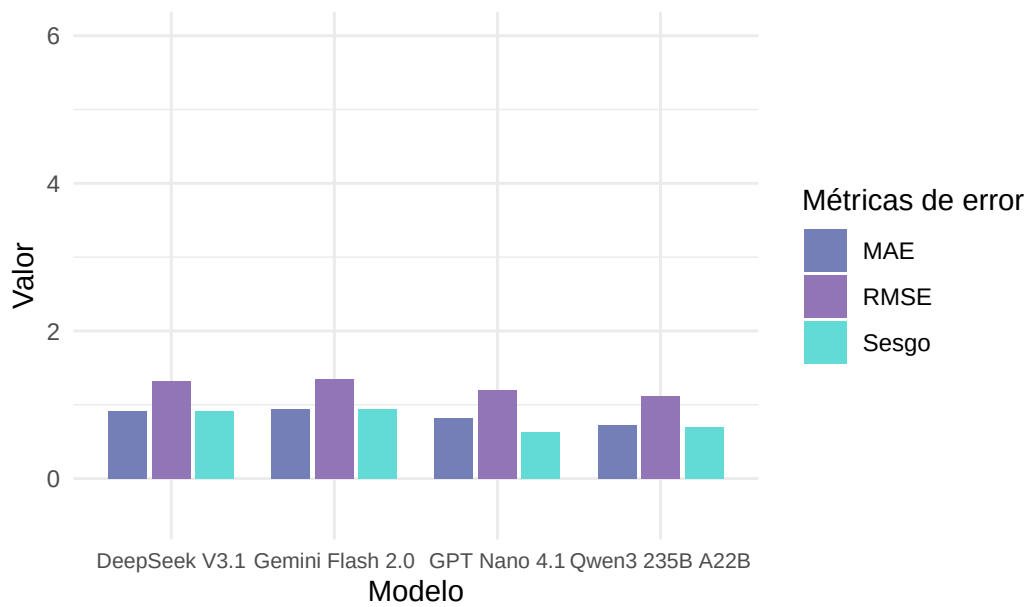
Contract: Voting Prompt: Basic Vulnerability: CPICheck



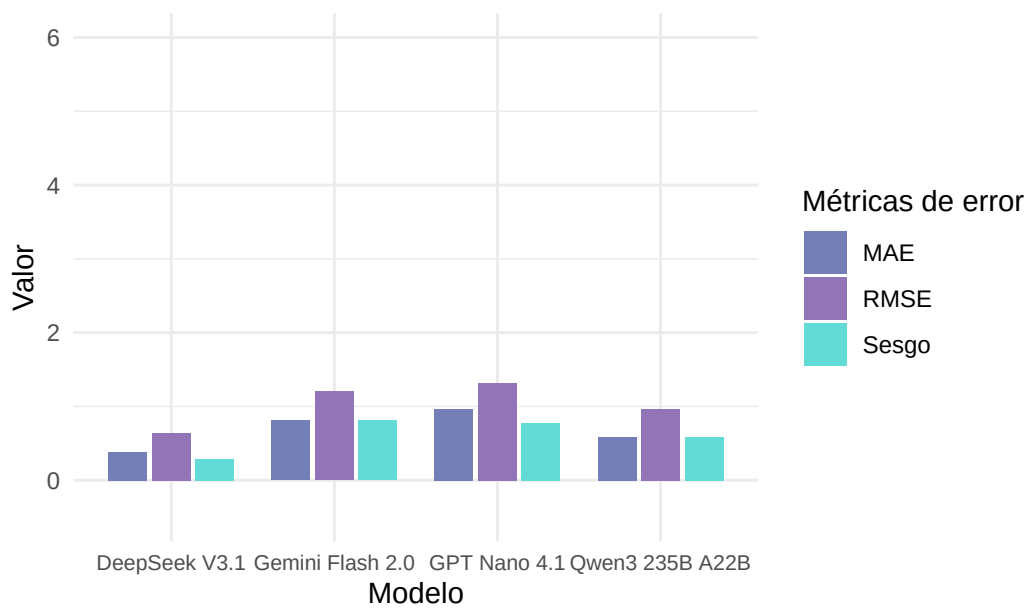
Contract: Voting Prompt: Complex Vulnerability: CPICheck



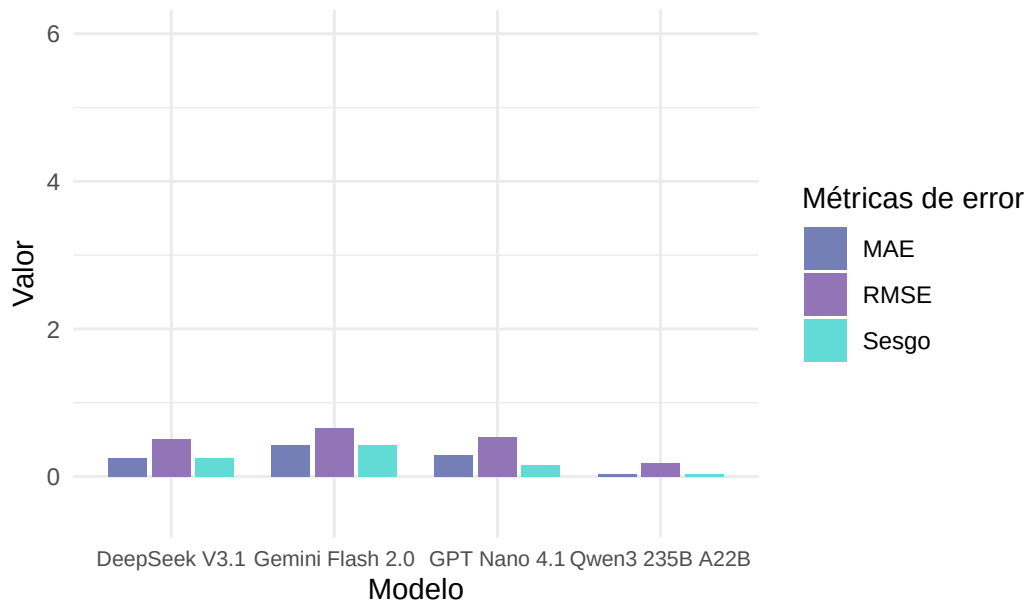
Contract: Voting Prompt: Basic Vulnerability: OverflowCheck



Contract: Voting Prompt: Complex Vulnerability: OverflowCheck



Contract: Voting Prompt: Basic Vulnerability: UnderflowCheck



Contract: Voting Prompt: Complex Vulnerability: UnderflowCheck

